Table 2: A simulation of how the developed library mitigates security attacks

| Attack category and scripts | Vulnerable setup (without Zero Trust) | Vulnerability exploited | Zero Trust defence mechanism | Security mitigation outcome |
|---|---|---|---|---|
| **Man-in-the-middle** mitm-attack.cc vs mitm-mitigation.cc | Sensor sends plaintext UDP packets (*TEMP:45.2*) to an on-path node on port 8080. Attacker alters payload to *TAMPERED: TEMP:45.2* and forwards it to gateway port 9090. | Unencrypted transmission and absence of endpoint identity verification allow on-path payload tampering. | • *CertificateAuthority::SignIdentity*<br>• *ZtPolicyEngine::AuthorizeWithCert*<br>• *ZtTlsHandshake::StartHandshake*<br>• *EncryptPayload* (AES-CBC + random IV) | Gateway validates session via HasSession and decrypts via DecryptPayload. Modified or unauthenticated packets cannot be decrypted and are immediately dropped. |
| **Identity spoofing** spoofing-attack.cc vs spoofing-mitigation.cc | Attacker connects directly to gateway TCP port 8080 posing as a valid sensor and pushes forged readings *(@$%!@ garbageDATA#)*. Gateway accepts and logs it. | Gateway accepts any incoming TCP connection without validating node certificates or digital signatures. | • *SecureSensorApp & SecureSink*<br>• *CertificateAuthority*<br>• *ZtPolicyEngine role verification (temp-sensor)* | Gateway mandates a valid CA-signed certificate before accepting connections. Attacker lacks a legitimate private key and signed certificate, causing immediate connection drop. |
| **Unauthorized access** unauthorized-access-attack.cc vs unauthorized-access-mitigation.cc | Rogue client connects to an open TCP sink on port 9090 and sends destructive commands like *DELETE ALL DATA*. Sink processes command without checks. | Flat network perimeter with no role-based access control or authorisation checks on backend endpoints. | • *ZtPolicyEngine::AddAuthorized*<br>• *ZtPolicyEngineWithCert*<br>• Revocation list check<br>• *ZtLogger::LogCertRejected* | Sink enforces strict identity authorisation and active session checks. Unauthorised attacker fails policy validation, is logged as rejected, and command execution is prevented. |

counter critical threats including man-in-the-middle attacks, identity spoofing, and unauthorised access. By combining static policy checks with dynamic trust scoring and automated revocation, the library delivers a practical, standard-aligned security framework for next-generation Internet of Things deployments END

**By: Arun Santhosh R.A. and Dr T. Subbulakshmi**

**Arun Santhosh** is a FOSS enthusiast with an interest in data security and backup.

**Dr T. Subbulakshmi** is a professor in the School of Computer Science and Technology at VIT Chennai, with research interests in data privacy, cloud computing, and intelligent systems.

## References

- Zero Trust IoT library documentation, *https://the-ns3-programmer.github.io/zero-trust-iot-library/*
- NIST SP 800-207 Standard (Zero Trust architecture), Scott Rose, Oliver Borchert, Stu Mitchell, Sean Connelly, National Institute of Standards and Technology Special Publication 800-207, August 2020, DOI: *https://doi.org/10.6028/NIST.SP.800-207*
- NS-3 App Store - Zero Trust IoT: App listing, release history, and installation requirements for the Zero Trust module for IoT in ns-3, Available at: *https://apps.nsnam.org/app/zero-trust-iot/*